Toegepaste Informatica

UCLL HOGESCHOOL

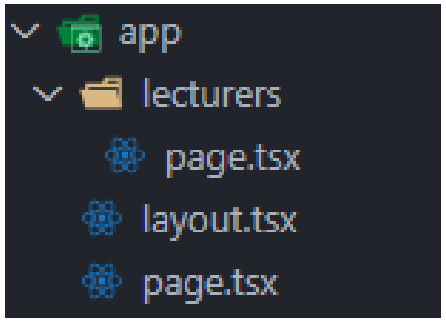
Full-stack software development

Front-end: dynamic routing

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

App router: recap

- A page is a React component which can be navigated to (= **routes**)
- Pages are associated with a route based on their file name
- For example
  - **app/page.ts** is associated with **/** route
  - **app/lecturers/page.ts** is associated with **/lecturers** route



App router: reserved filenames

- Each folder can have files with these reserved names (list is not exhaustive):
  - **page.tsx** is the main entry point for each folder.
  - **layout.tsx** contains the wrapping layout. If we don't provide one, the autogenerated layout.js in the root app folder is used.
  - **head.tsx** used to customize the head of the page - metatags, title etc.
  - **error.tsx** is used in data fetching.
  - **not-found.tsx** is used to throw a 404 error
  - **loading.tsx**

- https://nextjs.org/docs/app/api-reference/file-conventions for the full list

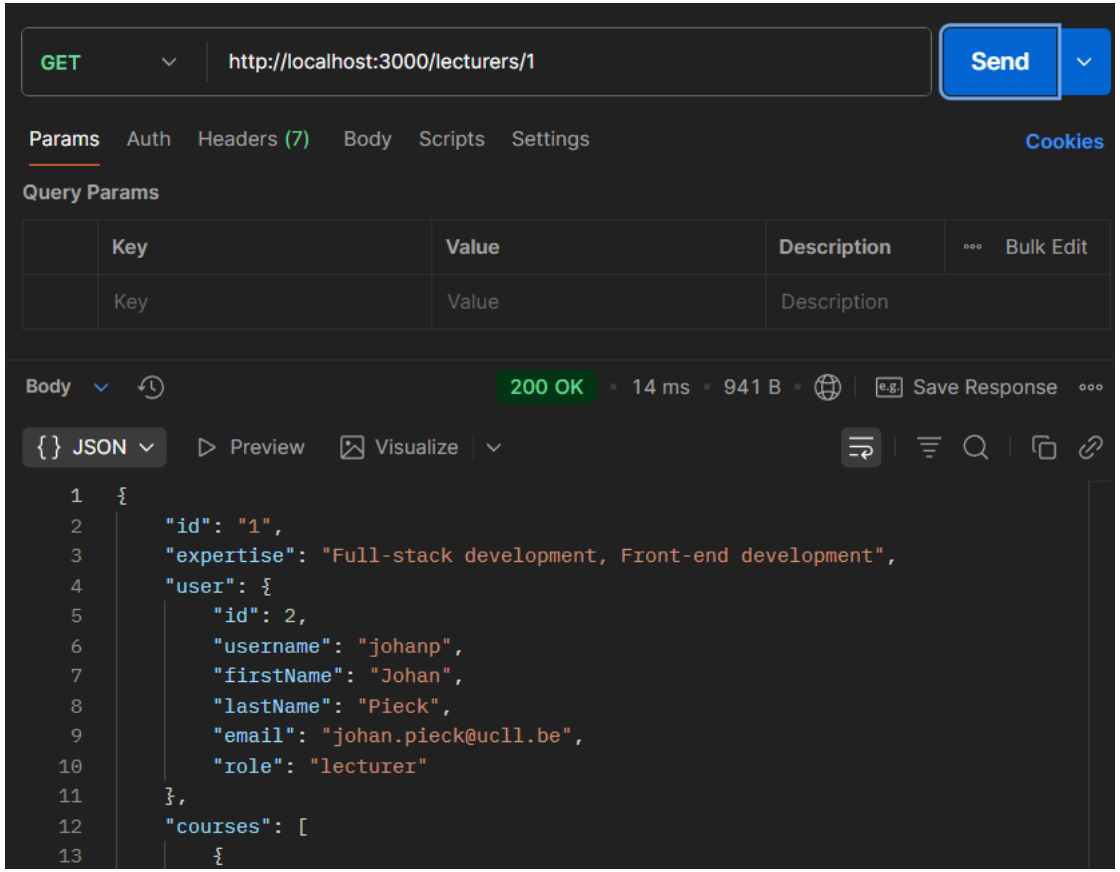
Recap: Routing

- Use the **Link** component of **next/link** to link between pages in your application
- <Link> allows you to do client-side navigation and accepts props that give you better control over the navigation behaviour
- Client-side navigation: page transition happens using **JavaScript**, which is faster than the default navigation done by the browser

```
const Header: React.FC = () => {
    return (
        <header>
            <a> Courses App</a>
            <nav>
                <Link href="/">Home</Link>
                <Link href="/lecturers">Lecturers</Link>
                <Link href="/schedule/overview">Schedules</Link>
            </nav>
        </header>
    );
};
```

Dynamic routing

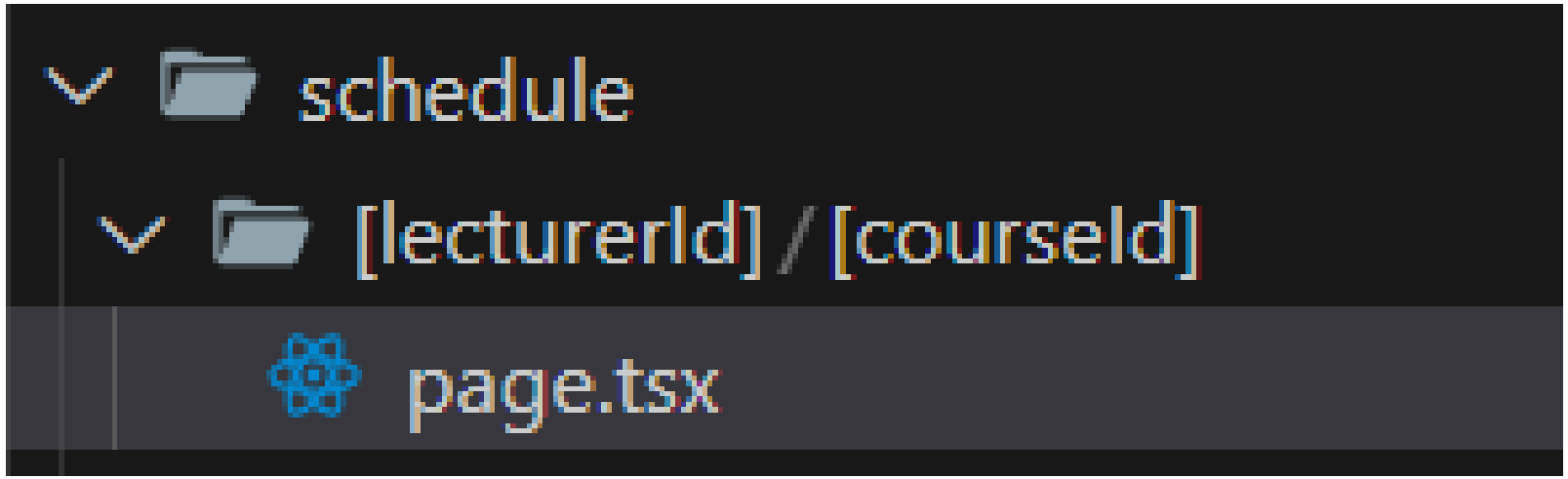
- Remember from back-end: we could add path parameters to an endpoint to refine our call.



Dynamic routing

- We can do the same with the App router:
  - The create schedule page: needs the id of the lecturer and the id of the course
  - we want that page to be available at this url /schedule/{lectorId}/{courseId}

  - To achieve this: wrap the folder names in [] to create a dynamic segment in a route.



Recap: Promises & async/await

- A **promise** represents a value that might not be immediately available but will be resolved at some point in the future.
  - In 1 of three states:
    - **Pending**: Initial state, neither fulfilled nor rejected
    - **Fulfilled**: Operation completed successfully
    - **Rejected**: Operation failed
- async/await is a clearer syntax for using promises:
  - An async method will always return a Promise
  - Await: pause execution until the Promise is fulfilled or throw an exception when it’s rejected.
  - Await can only happen inside of an async method

Dynamic routing: getting the params

```tsx
type PageProps = {
  params: Promise<{
    lecturerId: string;
    courseId: string;
  }>;       jhnpck, 3 weeks ago • Fully working …
};

type DataResult = {
  data: { lecturer: Lecturer; course: Course } | null;
  error: string | null;
};

const getData = async (
  lecturerId: string,
  courseId: string
): Promise<DataResult> => {
  try {
    const lecturerRes = LecturerService.getLecturerById(lecturerId);
    const courseRes = CourseService.getCourseById(courseId);

    const [lecturer, course] = await Promise.all([lecturerRes, courseRes]);
    return { data: { lecturer, course }, error: null };
  } catch (error) {
    return { data: null, error: (error as Error).message };
  }
};
```

The properties of the params as a **promise.**

A type to store the return from the server

An **async** function that gets the data from the server and returns it as a **promise**

await the fulfillment of both promises

Dynamic routing: getting the params

```tsx
const CreateSchedulePage = async ({ params }: PageProps) => {
  const paramsResolved = await params;
  const { data, error } = await getData(
    paramsResolved.lecturerId,
    paramsResolved.courseId
  );

  return (
    <>
      <h1>Create new schedule</h1>

      {error && (
        <div className="text-red-800" role="alert">
          {error}
        </div>
      )}

      {data && (
        <section>
          <ScheduleForm lecturer={data?.lecturer} course={data?.course} />
        </section>
      )}
    </>
  );
};

export default CreateSchedulePage;
```

Page is an **async** function

await the fulfillment of the params promise to get the resolved parameters.

We'll see this in more detail in the part about server components.

await the result of the get data function and destructure the result.